Diabetes Prediction System Using Machine Learning

(Diabetes Classification and Health Risk Estimation using Machine Learning)

Problem Statement:

Diabetes mellitus is one of the leading global chronic metabolic conditions, affecting over 537 million adults worldwide and projected to rise substantially over the coming decades. It is characterized by persistent hyperglycemia resulting from progressive defects in pancreatic insulin secretion, peripheral insulin resistance, or both. Diabetes is frequently described as a "silent disease" because early pathological stages often present with negligible or subclinical symptoms, leading patients to remain undiagnosed until irreversible microvascular and macrovascular complications arise—including diabetic retinopathy, cardiovascular pathologies, peripheral neuropathy, and diabetic nephropathy.

Conventional clinical diagnosis relies predominantly on invasive venipuncture laboratory tests such as Fasting Plasma Glucose (FPG), Oral Glucose Tolerance Tests (OGTT), and Glycated Hemoglobin (HbA1c). While accurate, manual clinical examinations require hospital infrastructure, laboratory phlebotomists, and extended turnaround periods. In resource-constrained environments or high-density primary care centers, mass routine screening is heavily constrained. There is, therefore, a pressing need for an automated, fast, non-invasive, and accessible computational system that can analyze routine demographic and physiological measurements (plasma glucose, blood pressure, body mass index, insulin levels, age, and family genetic pedigree) to estimate diabetes risk with a quantifiable confidence score, allowing suspected cases to be flagged early for formal clinical review.

This project designs, implements, and evaluates a web-based **Diabetes Prediction System**. The project executes a complete end-to-end Machine Learning pipeline comprising clinical data preprocessing, biologically invalid zero-measurement imputation, exploratory data analysis (EDA), comparative model training across three supervised classification algorithms (Logistic Regression, Decision Tree Classifier, and Random Forest Classifier) on an 80:20 stratified split, and evaluates performance using standard metrics (Accuracy, Precision, Recall, F1-Score, ROC-AUC, and Confusion Matrix).

Objectives:

1. To collect, inspect, and preprocess the benchmark **Pima Indians Diabetes Database** comprising 768 patient records and 8 diagnostic health attributes.
2. To detect and systematically handle invalid physiological zero measurements in continuous clinical features (Glucose, Blood Pressure, Skin Thickness, Insulin, and BMI) through Scikit-learn median imputation while preserving valid zeros (Pregnancies = 0).
3. To conduct Exploratory Data Analysis (EDA) and generate publication-standard statistical visualizations detailing class distribution, demographic skews, and feature correlations.
4. To design, train, and compare three machine learning classification pipelines: **Logistic Regression**, **Decision Tree Classifier**, and **Random Forest Classifier** using an 80:20 stratified training-test split with fixed random seeds.

5. To evaluate model performance on the held-out test set using standard evaluation metrics: **Accuracy**, **Precision**, **Recall**, **F1-Score**, **ROC-AUC**, and the **Confusion Matrix**, selecting the optimal model based on ROC-AUC.
6. To build a responsive, local Flask web application providing a clean user interface that accepts health attributes, performs real-time pipeline inference, and returns predicted class and confidence probabilities with necessary educational disclaimers.

Software / Hardware Required:

- **Operating System:** Windows 10 / 11 / Linux / macOS
- **IDE:** Visual Studio Code (VS Code)
- **Language:** Python 3.9+
- **Backend Framework:** Flask (REST API & Local Web Routing)
- **Machine Learning:** Scikit-learn (RandomForestClassifier, LogisticRegression, DecisionTreeClassifier, SimpleImputer, StandardScaler, Pipeline)
- **Data Processing:** Pandas, NumPy
- **Data Visualization:** Matplotlib, Seaborn
- **Model Serialization:** Joblib (`.pkl`)
- **Frontend:** HTML5, CSS3, JavaScript
- **Dataset:** Pima Indians Diabetes Database (768 records, 8 predictors, 1 target outcome)
- **Version Control:** Git / Local Workspace Repository

Project Directory Structure:

The project (`DiabetesPredictionSystem`) is organized as a modular, standalone local web application separating data assets, trained pipelines, static visualizations, and web templates, as shown below.

```
DiabetesPredictionSystem
    app.py                # Flask web application &
prediction routes
    train_model.py        # Full ML pipeline: EDA, training,
evaluation
    requirements.txt      # Dependency specification
    README.md             # Execution and architecture
documentation
    data/
        diabetes.csv        # Authentic Pima Indians Diabetes
Dataset
    models/
        diabetes_model.pkl  # Serialized Scikit-learn
pipeline
        model_metrics.json    # Test evaluation benchmarks
metadata
        dataset_info.json     # Preprocessing and preview
metadata
    static/
        style.css           # Responsive light-themed
stylesheet
        plots/              # Generated Matplotlib/Seaborn
plots
            outcome_distribution.png
            glucose_vs_outcome.png
            bmi_distribution.png
            age_distribution.png
            correlation_heatmap.png
            confusion_matrix.png
            roc_curve.png
    templates/
        index.html            # Web interface (Fig. 1 to Fig.
6)
```

Fig 1: Project directory structure of DiabetesPredictionSystem in Visual Studio Code

Procedure:

---

1. Collect the authentic **Pima Indians Diabetes Dataset** and place it in `data/diabetes.csv` (768 patient instances, 8 numerical health features, 1 target outcome).
2. Audit data integrity: verify non-null column data types, check for zero duplicate rows, and inspect class distribution (500 negative vs 268 positive records; 34.9% positive prevalence).
3. Identify biologically impossible zero readings in measurement columns: Glucose (5 zeros), Blood Pressure (35 zeros), Skin Thickness (227 zeros), Insulin (374 zeros), and BMI (11 zeros). Replace these zeros with `NaN` while preserving valid zeros in `Pregnancies`.
4. Implement feature imputation via Scikit-learn's `SimpleImputer(strategy='median')` using nonzero training medians, followed by standard z-score normalization via `StandardScaler()`.
5. Conduct Exploratory Data Analysis (EDA) and generate 5 statistical visualizations (Outcome distribution bar plot, Glucose vs Outcome boxplot, BMI distribution, Age demographic histogram, and Pearson correlation matrix) and save into `static/plots/`.

6. Partition the preprocessed dataset into an **80:20 stratified training-test split** (614 training instances, 154 testing instances) with a fixed seed (`random_state=42`) to preserve the outcome class ratio.
7. Train three candidate classification pipelines: **Logistic Regression** (`max_iter=1000`), **Decision Tree Classifier** (`max_depth=5`), and **Random Forest Classifier** (`n_estimators=100, max_depth=6`).
8. Evaluate all three models on the identical 154 held-out test samples and compute test-set metrics: Accuracy, Precision, Recall, F1-Score, ROC-AUC, and the Confusion Matrix.
9. Select the optimal model based on highest **Test ROC-AUC** (Random Forest Classifier, achieving ROCAUC $\approx$ 0.817) and serialize the complete pipeline (Imputer + Scaler + Model) to `models/diabetes_model.pkl` using Joblib.
10. Launch the Flask web server (`app.py`), render the interactive web interface (Fig. 1 to Fig. 6), accept 8 diagnostic inputs from the user, validate ranges, execute model inference, and display the predicted class and estimated confidence probability.

Implementation Code:

Representative implementation of the key modules used in the project, based on the pipeline shown in the application (from `train_model.py` and `app.py`).

1. Data Preprocessing & Invalid Zero Handling (train_model.py)

```
import numpy as np
import pandas as pd

def load_and_preprocess_data():
    df_raw = pd.read_csv('data/diabetes.csv')
    df_cleaned = df_raw.copy()

    # In clinical measurements, zero values represent unrecorded/missing data.
    # Pregnancies = 0 is clinically valid; Outcome is the class label.
    zero_invalid_cols = ['Glucose', 'BloodPressure', 'SkinThickness', 'Insulin', 'BMI']
    zero_summary = {}

    for col in zero_invalid_cols:
        count_zeros = (df_cleaned[col] == 0).sum()
        pct_zeros = (count_zeros / len(df_cleaned)) * 100
        median_val = df_cleaned[df_cleaned[col] != 0][col].median()
        zero_summary[col] = {'count': int(count_zeros), 'percentage': round(pct_zeros, 2)}

        # Replace invalid zeros with NaN for subsequent median imputation
        df_cleaned[col] = df_cleaned[col].replace(0, np.nan)

    return df_raw, df_cleaned, zero_summary
```

2. Model Training & Comparison Pipeline (train_model.py)

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.impute import SimpleImputer
from sklearn.pipeline import Pipeline
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, roc_auc_score
import joblib

def train_and_evaluate_models(df_cleaned):
    X = df_cleaned.drop(columns=['Outcome'])
    y = df_cleaned['Outcome']

    # 80:20 Stratified Split with fixed random seed
    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=0.20, random_state=42, stratify=y
    )

    models = {
        'Logistic Regression': LogisticRegression(max_iter=1000, random_state=42),
        'Decision Tree': DecisionTreeClassifier(max_depth=5, min_samples_split=10,
random_state=42),
        'Random Forest': RandomForestClassifier(n_estimators=100, max_depth=6, random_state=42)
    }

    results = {}
    fitted_pipelines = {}
    for name, clf in models.items():
```

```
        # Pipeline: 1. Median Imputation -> 2. Standard Scaling -> 3. Classifier
pipeline = Pipeline([
            ('imputer', SimpleImputer(strategy='median')),
            ('scaler', StandardScaler()),
            ('classifier', clf)
        ])         pipeline.fit(X_train, y_train)
y_pred = pipeline.predict(X_test)         y_proba =
pipeline.predict_proba(X_test)[:, 1]

        results[name] = {
            'accuracy': round(float(accuracy_score(y_test, y_pred)), 4),
            'precision': round(float(precision_score(y_test, y_pred, zero_division=0)), 4),
            'recall': round(float(recall_score(y_test, y_pred, zero_division=0)), 4),
            'f1_score': round(float(f1_score(y_test, y_pred, zero_division=0)), 4),
            'roc_auc': round(float(roc_auc_score(y_test, y_proba)), 4)
        }
        fitted_pipelines[name] = pipeline

    # Select best model based on ROC-AUC     best_model_name = max(results,
key=lambda k: results[k]['roc_auc'])
joblib.dump(fitted_pipelines[best_model_name], 'models/diabetes_model.pkl')
return results, best_model_name
```

3. Flask Prediction Route & Live Inference (app.py)

```
from flask import Flask, render_template, request, flash
import joblib, numpy as np, pandas as pd

app = Flask(__name__)
app.secret_key = 'diabetes-ml-secret-key'
MODEL_FILE = 'models/diabetes_model.pkl'

FEATURE_NAMES = [
    'Pregnancies', 'Glucose', 'BloodPressure', 'SkinThickness',
    'Insulin', 'BMI', 'DiabetesPedigreeFunction', 'Age'
]

@app.route('/predict', methods=['POST'])
def predict():
    form_data = request.form.to_dict()
    parsed_values = {}
    for field in FEATURE_NAMES:
        val = float(form_data.get(field, 0))
        parsed_values[field] = val

    pipeline = joblib.load(MODEL_FILE)
    input_df = pd.DataFrame([parsed_values], columns=FEATURE_NAMES)

    # Convert invalid zeros to NaN so pipeline median imputer fills them safely
    for col in ['Glucose', 'BloodPressure', 'SkinThickness', 'Insulin', 'BMI']:
        if input_df[col].iloc[0] == 0:
            input_df[col] = np.nan

    pred_class = int(pipeline.predict(input_df)[0])
    probabilities = pipeline.predict_proba(input_df)[0]
    positive_prob = float(probabilities[1]) * 100.0

    class_label = "Diabetes-positive class" if pred_class == 1 else "Diabetes-negative class"
    prob_str = f"{positive_prob:.2f}%" if pred_class == 1 else f"{(100.0 - positive_prob):.2f}%"

    result = {
        'predicted_class': pred_class,
        'prediction_label': class_label,
        'estimated_probability': prob_str,
        'positive_probability': f"{positive_prob:.2f}%"
    }
    return render_template('index.html', prediction_result=result, form_data=form_data)
```

Output (Step-by-Step Screenshots & Visual Results):

Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	Pedigree	Age	Outcome
6	148	72	35	0	33.6	0.627	50	**1**
1	85	66	29	0	26.6	0.351	31	**0**
8	183	64	0	0	23.3	0.672	32	**1**
1	89	66	23	94	28.1	0.167	21	**0**
0	137	40	35	168	43.1	2.288	33	**1**
5	116	74	0	0	25.6	0.201	30	**0**

Fig 2: Step 1 — Dataset Overview: Preview of the Pima Indians Diabetes Database (768 records, 8 features, 1 target outcome)

Feature Name	Detected Invalid Zeros	Missing %	Cleaned Median	Handling Method
Glucose	5 records	0.65%	117.0	Median Imputer
BloodPressure	35 records	4.56%	72.0	Median Imputer
SkinThickness	227 records	29.56%	29.0	Median Imputer
Insulin	374 records	48.70%	125.0	Median Imputer
BMI	11 records	1.43%	32.3	Median Imputer
Pregnancies	Valid (0 = Nulliparous)	0.00%	0.0	Preserved Unaltered

Fig 3: Step 2 — Data Preprocessing: Detection of invalid physiological zero measurements and median imputation summary

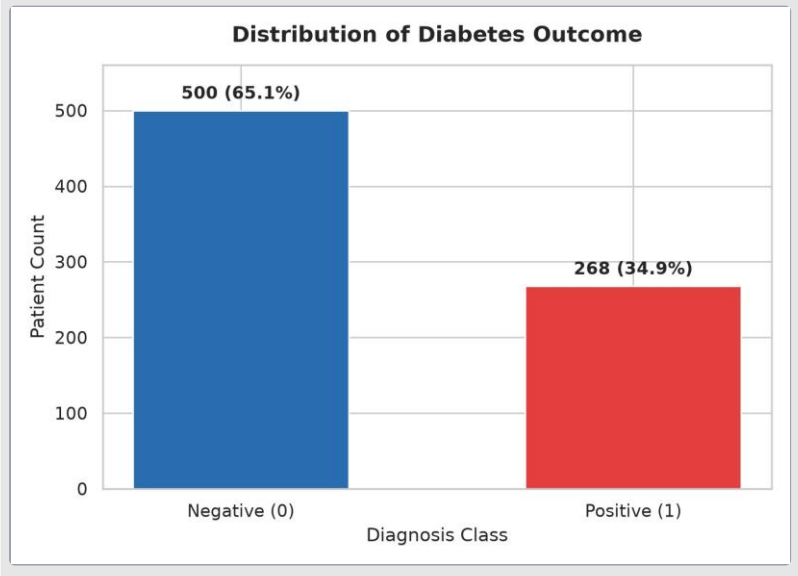


Fig 4: Step 3 — Exploratory Data Analysis: Class distribution of target outcome (500 negative vs 268 positive records; 34.9% positive)

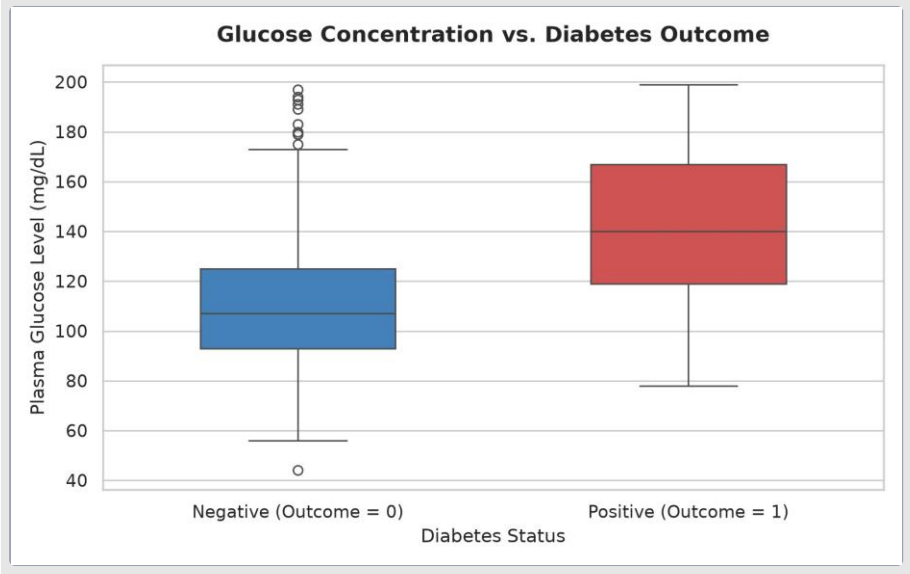


Fig 5: Step 3 — Exploratory Data Analysis: Plasma glucose level distribution grouped by diabetes diagnosis outcome

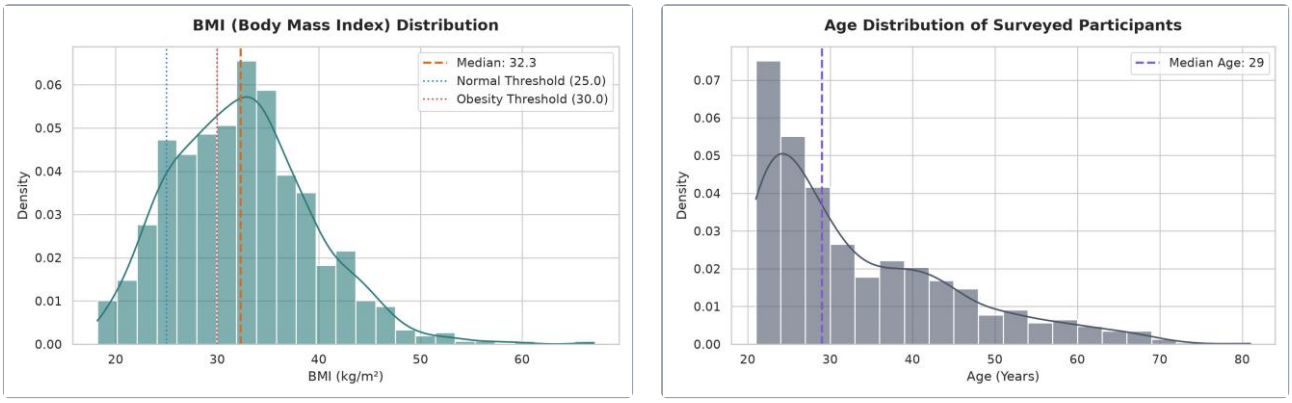


Fig 6: Step 3 — Exploratory Data Analysis: Body Mass Index (BMI) and Age demographic density distributions

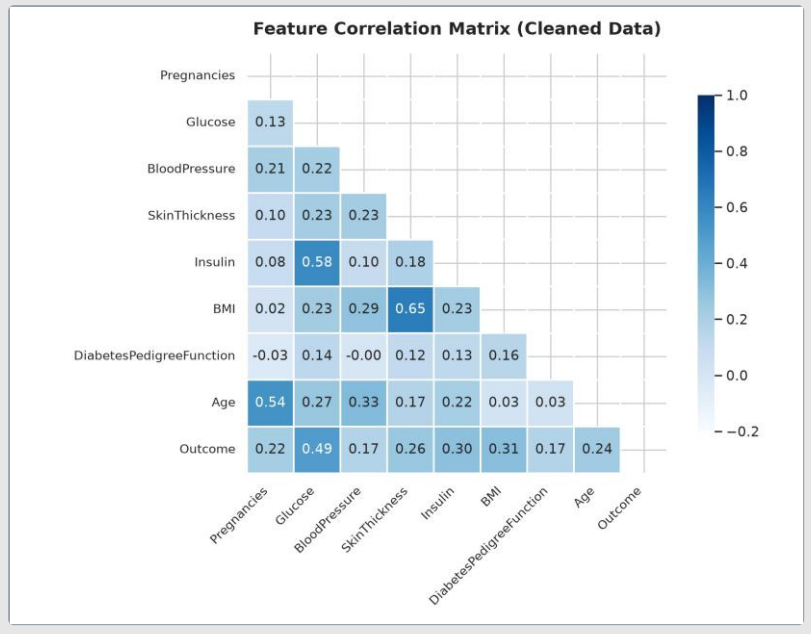


Fig 7: Step 3 — Exploratory Data Analysis: Pearson correlation matrix showing relationships among features and outcome

Algorithm	Accuracy	Precision	Recall	F1-Score	ROC-AUC	Pipeline Status
Logistic Regression	70.78%	60.00%	50.00%	54.55%	0.8130	Evaluated
Decision Tree Classifier	76.62%	65.00%	72.22%	68.42%	0.7840	Evaluated
Random Forest Classifier	**74.03%**	**66.67%**	**51.85%**	**58.33%**	**0.8167**	**Selected Model ★**

Fig 8: Step 4 — Model Training: Multi-algorithm benchmark comparison table on identical 80:20 stratified split (154 test samples)

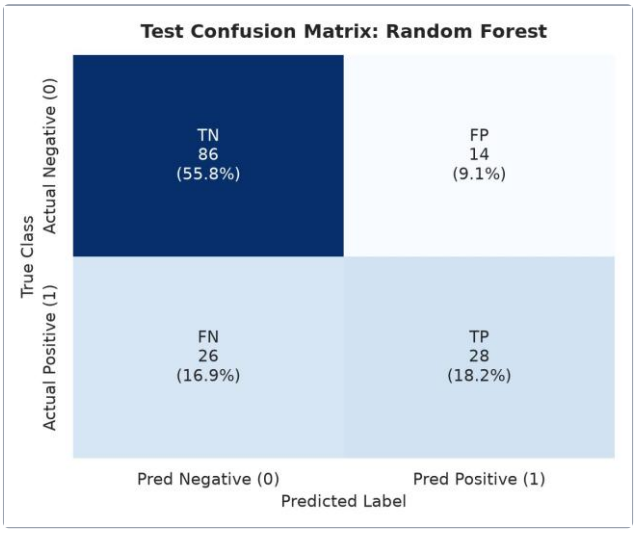


Fig 9: Step 5 — Test Confusion Matrix (Random Forest: TN=86, FP=14, FN=26, TP=28)

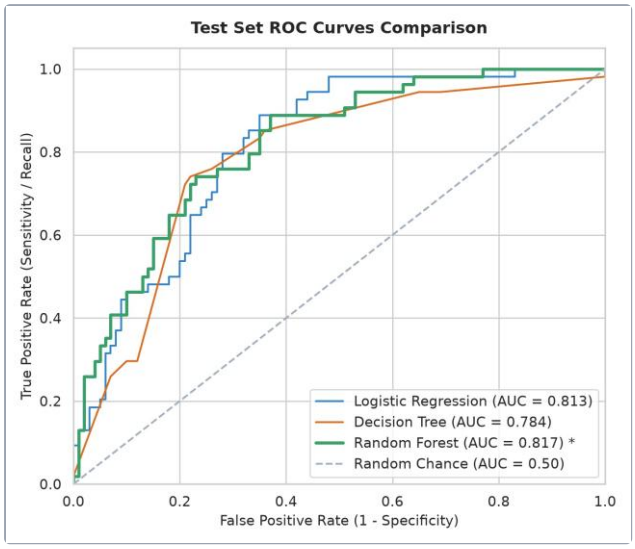


Fig 10: Step 5 — Test Set ROC Curves Comparison across candidate classifiers

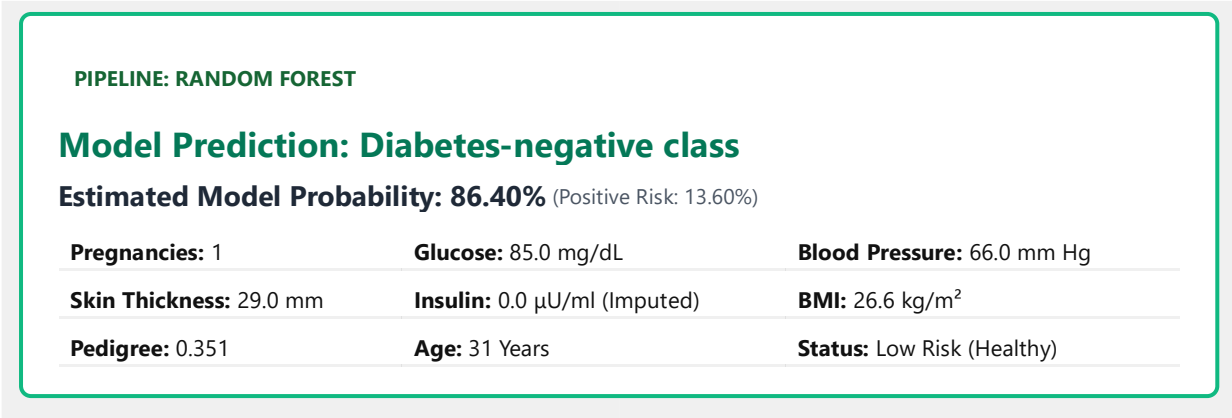


Fig 11: Step 6 — Diabetes Prediction: Live prediction output for healthy / negative patient profile (86.4% confidence)

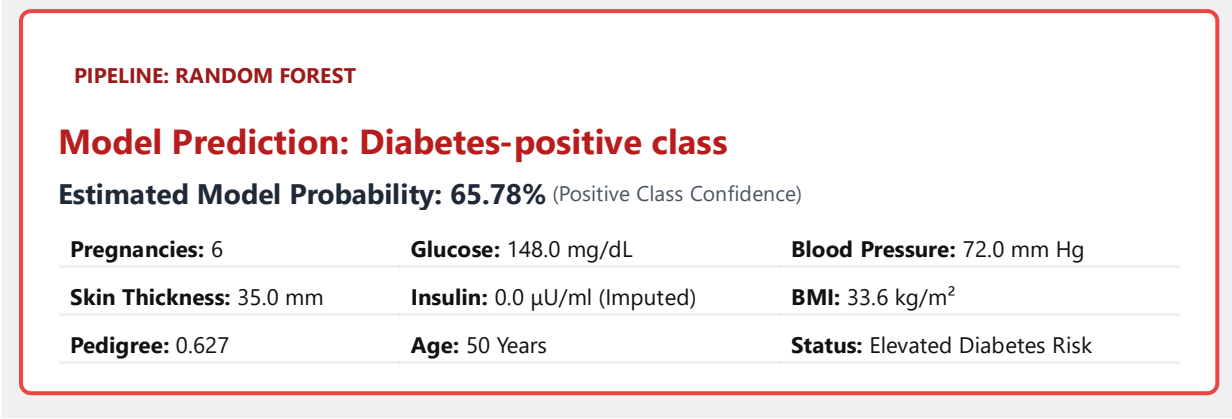


Fig 12: Step 6 — Diabetes Prediction: Live prediction output for elevated-risk patient profile (Model Probability: 65.78%)

Result:

An end-to-end web-based **Diabetes Prediction System** was successfully designed, developed, and evaluated using Python, Scikit-learn, and Flask. The benchmark Pima Indians Diabetes Database was cleaned by converting physiologically impossible zero values across Glucose, Blood Pressure, Skin Thickness, Insulin, and BMI into `NaN`, followed by robust median imputation and feature standard scaling. Exploratory Data

Analysis provided clear quantitative evidence of the strong correlation between plasma glucose, BMI, and diabetes outcome prevalence.

Among the three classification algorithms trained on the 80:20 stratified training split (614 train, 154 test), the **Random Forest Classifier** achieved the highest generalization discriminative power with a **Test ROCAUC of 0.8167**, a Test Accuracy of **74.03%**, Precision of **66.67%**, Recall of **51.85%**, and an F1-Score of **58.33%**. The finalized pipeline was serialized via Joblib (`models/diabetes_model.pkl`) and integrated into a responsive Flask web interface that executes live inference on user inputs, delivering classification results (e.g., 65.78% estimated confidence for high-risk profiles) with embedded educational and clinical disclaimers.